UNMARK

Tone-Factored Input Adaptation for
Diacritic-Robust Vietnamese Language Understanding

Research Proposal

Version 1.3 — 19 August 2026

Author	Quoc-Linh Vo (Võ Quốc Linh)
Affiliation	Faculty of Information Technology University of Science, VNU-HCM (APCS)
Contact	vqlinh2433@apcs.fitus.edu.vn
Status	Active — specification lock, then feasibility gates
Venue	Deliberately undecided. The venue is chosen after the results are in, not before. Nothing in this document is shaped by a submission deadline.

This document is the working specification for the project. It records the idea, its exact position relative to prior work, the method, the experimental design, the implementation plan, and the decision gates at which the project should be abandoned or redirected.

Executive summary

Vietnamese users routinely type without diacritics, yet every pretrained Vietnamese language model is trained and evaluated on fully diacritized text. The standard responses are to *restore* the missing marks at the string level, or to *measure* how much performance degrades when they are absent.

UNMARK asks a different question: can a frozen pretrained Vietnamese encoder be adapted to operate *directly* under structured orthographic information loss — without restoring the text, without changing the tokenizer, and without updating the encoder?

The proposed answer is a small input-side module that re-expresses Vietnamese orthography as structured channels: an unmarked base channel, a tone channel, and a letter-diacritic channel. The design is inspired by multi-source partial-diacritic modelling for Arabic, but Vietnamese breaks a core assumption of that formulation. In Arabic, the absence of a diacritic reliably signals *unavailable information*. In Vietnamese it does not: the *ngang* tone is itself written with no mark at all, so an unmarked syllable is simultaneously a valid observation and a possible information loss. UNMARK therefore models an UNMARKED state that is deliberately ambiguous, and lets the encoder resolve it from context.

The evaluation is organised around a question that has not been answered for Vietnamese: *at which level should structured orthographic loss be handled* — the string, the input representation, or the output representation? Three interventions are compared under one backbone, one task suite, and one controlled corruption protocol.

Contribution type. This is a *combination* contribution plus a new problem framing, not a new mechanism. Section 3 states explicitly what is and is not claimed. Where the work is eventually submitted is a decision to make once the results exist; it is not a premise of the design, and it must not be allowed to inflate the claims.

Contents

1 Background and motivation

1.1 Vietnamese tone orthography

Vietnamese is written in a Latin-based script (*Quốc ngữ*) in which each syllable follows a fixed phonotactic template C_1wVC_2T: onset, medial, nucleus, coda, and tone. The inventory is closed and small — on the order of 21 onsets, 155 finals, and 6 tones — yielding roughly 7,000 syllables in practical use out of about 19,000 that are phonotactically possible. This regularity means a Vietnamese syllable can be decomposed into its components by rule, deterministically, with no model and no lookup table.

Two distinct classes of diacritic must be separated, because they carry different information and can be lost independently:

- **Tone marks** realise five of the six lexical tones: *sắc*, *huyền*, *hỏi*, *ngã*, *nặng*.
- **Letter diacritics** form distinct vowel and consonant letters: ă, â, ê, ô, ơ, ư, đ. These belong to the letter identity, not to the tone.

Tone is lexically contrastive: changing the tone changes the word, not its register or nuance. A single base syllable can correspond to several unrelated words.

1.2 The unmarked-tone ambiguity

The sixth tone, *ngang*, is written by placing *no mark at all*. This is the linguistic fact on which the whole project turns.

Consequently, when a system observes a syllable carrying no tone mark, the orthography alone does not distinguish two very different situations:

Situation	Meaning of "no tone mark"
Genuine *ngang*	The tone *is ngang*. Information is present.
Stripped input	The tone was one of five others; the mark was lost.

Both surface as an identical string. The ambiguity is *not observable* at inference time, and no amount of engineering removes it — it is a property of the writing system.

This is where Vietnamese departs from the languages in which partial-diacritic modelling was developed. In Arabic, ordinary text is written undiacritized, so absence of a mark means "not yet specified" — a single, unambiguous state that a dedicated blank symbol represents faithfully. In Vietnamese, the same symbol would conflate a meaningful phonological category with an information loss.

1.3 Why this matters in deployment

Undiacritized Vietnamese is not an edge case. It is how a large share of real-world Vietnamese input is produced: search boxes, chat, comments, quick notes, and any setting where typing speed matters. It also arises mechanically, as OCR systematically drops tone marks — reported error taxonomies for Vietnamese scene text place tone-drop at roughly a fifth of all recognition errors, rising further when recogniser confidence is low.

Meanwhile, the pretrained models used to process this input were trained on Wikipedia and news text, which is fully diacritized. Prior work has already established that Vietnamese PLM performance degrades substantially as diacritics are removed, and that subword tokenizers fragment undiacritized Vietnamese badly. The distribution mismatch is documented; what is missing is an inexpensive way to *operate* under it.

1.4 Why existing responses are insufficient

String-level restoration. Predict the missing marks, then encode the repaired string. Vietnamese diacritic restoration is a mature area with high reported token-level accuracy. Two concerns remain untested. First, restoration must *commit* to a single output; where the preimage is genuinely ambiguous, a wrong commitment is unrecoverable downstream. Second, reported accuracies are measured on well-formed running text; short, context-poor inputs are a different distribution. Whether high token accuracy translates into recovered downstream performance is an open empirical question — and answering it is part of this project.

Output-level representation alignment. Encode the corrupted string as usual, then learn a light transformation pulling the resulting vector towards the clean one, with the encoder frozen. This family is well established for typographical noise. It repairs the consequence after tokenization has already fragmented the input.

Retraining. Pretrain a model on data that includes undiacritized text. Effective, and demonstrably improves robustness — but it costs a pretraining run, and it does not help the many teams already running an existing encoder in production.

UNMARK occupies the remaining position: intervene at the *input representation*, before the encoder, leaving the encoder untouched.

2 Research question and hypotheses

Primary research question

RQ. Can a pretrained Vietnamese language model be made robust to structured orthographic information loss without restoring the text, changing the tokenizer, or updating the pretrained encoder?

Secondary research question

RQ′. At which level should structured orthographic loss be handled: the string, the input representation, or the output representation?

Hypotheses

H1 (architectural feasibility). A frozen pretrained encoder accepts a synthesised input embedding built from orthographic channels without substantial loss on fully diacritized input.

H2 (residual gap). Even with a strong off-the-shelf diacritic restorer, a measurable downstream gap remains between restored input and fully diacritized input.

H3 (input-level advantage). Channel-factored input adaptation recovers a larger fraction of that gap than output-level representation alignment, at comparable or lower parameter and latency cost.

H4 (ambiguity modelling). Explicitly modelling the UNMARKED state as ambiguous outperforms treating it as a single "missing" symbol, and outperforms treating it as a confident *ngang*.

H2 is the load-bearing hypothesis: if it fails, the project has no method-paper story. H4 is the hypothesis carrying whatever originality the work has. Both are tested early (Section 7).

3 Related work and positioning

3.1 The five nearest lines of work

Multi-source partial-diacritic modelling (Arabic). Bahar et al. (2023) propose a two-source model in which an undiacritized character stream and a diacritic stream are embedded separately

and summed, with a blank symbol for positions carrying no diacritic information, trained with random masking of the diacritic channel at varying rates. This is the closest ancestor of UNMARK. Its *output* is diacritics; UNMARK's output is a semantic representation. Its blank symbol is unambiguous; UNMARK's UNMARKED state is not.

Vietnamese PLMs and diacritic degradation. ViSoBERT (2023) pretrains a Vietnamese social-media encoder and, as part of its analysis, measures degradation of Vietnamese PLMs under 25/50/75/100% diacritic removal across several classification tasks, and documents tokenizer fragmentation on undiacritized text. This is the empirical baseline UNMARK builds on — it establishes the problem, and it means degradation curves are not by themselves a contribution.

Deterministic Vietnamese phonological decomposition. PhonoSTFG (2026) performs NFD normalisation, tone extraction, tone stripping, and rule-based syllable parsing for Vietnamese, and uses the result as pairwise binary features forming an attention bias inside a graph, on top of a frozen PhoBERT with a per-dimension gated fusion of two streams. The decomposition procedure is reused here. The use is different: pairwise comparison between tokens versus per-token state representation at the input layer.

Factorized orthographic input representations. Work on decomposing accented characters into base plus modifier for machine translation across many languages, and recent character-level rich embeddings intended as drop-in replacements for subtoken embeddings, both argue that factorisation mitigates sparsity caused by subword tokenization. UNMARK inherits this argument and must not restate it as new.

Noise-robust dense retrieval. Contrastive alignment of noisy-query representations to clean ones with a frozen index, ranking-distillation approaches, and multi-positive variants. These operate at the output of the encoder and form the principal competing baseline.

3.2 What is *not* claimed

Stating these plainly in the paper is deliberate. A reviewer who sees the boundaries drawn honestly is more likely to accept what lies inside them.

- **N1.** Not the first to decompose a syllable into base and diacritic components.
- **N2.** Not the first to treat diacritics as a separate input channel that may be absent.
- **N3.** Not the first to train with randomly masked diacritics at multiple rates.
- **N4.** Not the first to measure Vietnamese downstream degradation under diacritic removal.
- **N5.** Not claiming that subword fragmentation is eliminated. UNMARK makes the diacritized and undiacritized forms share a base sequence; it does not make that sequence well tokenized.
- **N6.** Not claiming cross-lingual transfer to other diacritic-bearing orthographies. The formulation is potentially portable; validation is left to future work.
- **N7.** Not claiming Vietnamese is the "most extreme" case of orthographic ambiguity. It is an *informative* case, for the specific reason given in Section 1.2.

3.3 What *is* claimed

- **C1. A reframed objective.** Prior work on diacritics either restores them or measures the damage from their absence. UNMARK adapts a pretrained model to operate under the loss, targeting representation stability rather than orthographic recovery.
- **C2. Ambiguity-aware channel state.** The tone channel encodes only what is observable. UNMARKED deliberately conflates genuine *ngang* with stripped tone, and the training signal — in which *ngang* syllables are invariant under corruption while others are not — teaches the model that this state is ambiguous and must be resolved from context. This asymmetry has no analogue in the Arabic formulation.
- **C3. Parameter-efficient retrofit.** A reusable module for existing frozen Vietnamese encoders: no pretraining, no tokenizer change, no index rebuild.

C4. A three-level intervention comparison. String-level restoration, input-level factorisation, and output-level alignment, evaluated under one backbone, one task suite, one corruption protocol.

3.4 Positioning sentence for the paper

> *Existing work either restores missing diacritics or measures the degradation their removal causes. We instead ask whether a pretrained model can be adapted to operate directly under structured orthographic missingness. Our design is inspired by multi-source partial-diacritic modelling such as 2SDiac, but Vietnamese introduces a fundamentally different ambiguity: the absence of a tone mark simultaneously encodes the phonologically meaningful ngang tone.*

4 Method

4.1 Notation

Let x be a fully diacritized Vietnamese string and $\tilde{x}_p$ a corrupted version in which tone marks are removed from a fraction p of syllables. Let E_θ be a pretrained encoder with frozen parameters θ, Emb_θ its input embedding table, and $h(\cdot)$ the representation it produces (pooled or per-token as the task requires). Let A_ϕ denote the UNMARK module with trainable parameters ϕ, $|\phi| \ll |\theta|$.

4.2 Deterministic decomposition

A rule-based function maps a string to three parallel syllable-level streams:

$$\mathrm{dec}(x) \;=\; \big(\, b(x),\; \tau(x),\; \lambda(x) \,\big)$$

where b is the base form with all diacritics removed, τ is the tone state, and λ is the letter-diacritic state. The procedure is: Unicode NFD normalisation; separation of combining marks into tone marks (U+0300, U+0301, U+0303, U+0309, U+0323) and letter-forming marks (U+0302, U+0306, U+031B, plus the đ stroke); recomposition of the base.

Invertibility, stated precisely. An earlier version of this document required $\mathrm{rec}(\mathrm{dec}(x)) = x$ byte-for-byte. That requirement was not achievable under the channel design it specified, and the design — not the requirement — was the error.

Two facts settle the matter. First, a syllable may carry *several* letter-forming marks at different positions, so a single per-syllable letter state cannot reconstruct it. The letter-diacritic channel is therefore defined **per character**, not per syllable (Section 4.3). With that change, letter diacritics reconstruct exactly.

Second, one genuine ambiguity remains: Vietnamese admits two accepted positions for a tone mark over a vowel cluster (*hoà* versus *hòa*), and the same string may arrive in NFC or NFD form. These are orthographic variants that carry no semantic distinction, so dec canonicalises them deliberately.

The requirement is therefore:

$$\mathrm{rec}(\mathrm{dec}(x)) \;=\; \mathrm{canon}(x)$$

where canon applies Unicode NFC and a fixed tone-placement rule, and *every* difference between x and $\mathrm{canon}(x)$ is enumerable and logged. No silent loss is tolerated. Canonicalisation is not a limitation hidden in a footnote: it becomes the VARIANT evaluation condition (Section 6.3).

4.3 Channel state sets

Channel	Granularity	States
Base b	character	fully stripped letters; tokenized by the frozen tokenizer
Tone τ	**syllable**	5 marked tones (*sắc*, *huyền*, *hỏi*, *ngã*, *nặng*) + 2 policy slots — **7 states**
Letter λ	**character**	{NONE, breve, circumflex, horn, stroke, circumflex+?, ...} — small closed set
Non-Vietnamese	—	N/A in both channels; membership decided by the rule below

Deciding what counts as Vietnamese. Digits, punctuation and symbols are trivially N/A. Alphabetic spans are not: an undiacritized ASCII string may be an English word, a loanword, or a Vietnamese syllable that has simply lost its marks — and several strings are simultaneously a valid Vietnamese syllable and a valid English word. No deterministic rule resolves this, and the decomposition is required to be rule-based.

The rule is therefore chosen for determinism, not for correctness:

> An alphabetic span is treated as a **Vietnamese candidate** if it matches the Vietnamese syllable inventory after stripping; otherwise both channels are N/A.

Ambiguous spans are resolved towards Vietnamese, and this is documented as a known and deliberate error mode rather than hidden. One property matters more than the rule's accuracy: it is a pure function of the *stripped* form, so it assigns the same labels to clean and corrupted input and cannot break grid invariance.

Why the two channels have different granularity. Tone is a property of the *syllable*: one syllable carries exactly one tone, regardless of how many vowels it contains. Letter diacritics are properties of *individual letters*, and one syllable may carry several of them at once, on different characters. Putting both at syllable level, as an earlier draft did, makes the letter channel lossy and breaks reconstruction. This asymmetry is not a detail — it is what makes G0 passable.

Seven tone slots, so that all three H4 policies share one architecture. The tone table has 7 rows for every policy: five marked tones plus two slots whose meaning is set by the policy (Section 6.7).

Policy	Slot A	Slot B
OBSERVABLE (UNMARK)	UNMARKED	unused
FORCED-NGANG	*ngang*	unused
ORACLE	*ngang*	MISSING

A six-state table, as specified in v1.2, cannot express the oracle policy, which needs *ngang* and MISSING simultaneously. Giving all three policies an identical $7 \times d$ table removes any objection that the oracle was granted extra capacity.

Locked decision: no separate MISSING state at inference. An earlier draft used six tones plus a MISSING symbol. This is not implementable: at inference the system observes only *no visible mark*, and cannot know whether a mark was deleted. Any state set requiring that knowledge works only in the laboratory.

The tone channel therefore encodes *only what is observable*. Genuine *ngang* and stripped tone both map to UNMARKED. This conflation is intentional and is the technical core of the proposal:

- Under corruption, a *ngang* syllable is *invariant*; a toned syllable transitions to UNMARKED. The training distribution therefore contains UNMARKED tokens of both origins, in a ratio that varies with the corruption rate.
- The module cannot resolve the ambiguity from the channel value alone. It must learn that UNMARKED is a low-information state whose interpretation depends on the surrounding context.
- This is precisely the behaviour a blank-symbol formulation does not need to learn, because in the source language the blank is unambiguous.

4.4 Alignment between streams

This is the most error-prone engineering detail and must be settled before any training run.

Three streams do not share an index. The original string, the base string, and the syllable-level channel labels tokenize differently: $|T(x)| \neq |T(b(x))|$ in general, and one syllable may span several subwords.

The specification is:

1. The **base stream** defines the token grid. All positions are indexed by $T(b(x))$.
2. Each subword is assigned the channel labels of the syllable it belongs to, by tracking character offsets through tokenization.
3. The **tone label** of a syllable is copied to every subword that overlaps that syllable's character span.
4. The **letter labels** are per character. A subword spanning several characters pools in *embedding space*, not in label space:

$$l_i \;=\; \mathrm{Pool}\big(\{\, W_\lambda[\lambda_c] \;:\; c \in \mathrm{span}(\mathrm{token}_i) \,\}\big)$$

   with mean pooling for v1. Collapsing several characters back into one categorical label ("first mark wins") would reintroduce, at subword level, exactly the information loss that moving the channel to character level was meant to remove. Learned attention pooling is an ablation.
5. The **original stream** is *not* used as a third parallel stream, because it cannot be aligned to the base grid without an ad-hoc heuristic. If a residual path to the original embedding is wanted, it is added only in the fully diacritized condition, where $b(x)$ and x share the syllable segmentation.

Invariants to assert in unit tests, before any training run: the three label sequences have equal length; that length equals $|T(b(x))|$; every non-Vietnamese subword carries N/A in both channels; and corrupting the input changes the tone labels but never the base ids.

4.5 Fusion and gating

For each position i on the base grid:

$$e_i \;=\; \mathrm{Emb}_\theta\big(b_i\big), \qquad t_i \;=\; W_\tau[\tau_i], \qquad l_i \;=\; W_\lambda[\lambda_i]$$

$$f_i \;=\; \mathrm{LN}\big(W_f\,[\,e_i;\, t_i;\, l_i\,] + c_f\big), \qquad W_f \in \mathbb{R}^{d \times 3d}$$

Linear, not MLP. v1.2 specified a two-layer MLP in the architecture lock while costing a single linear projection in the parameter budget — the two did not agree. The main method is a **single linear projection**, which is what the budget in Section 4.7 describes and what keeps the parameter count matched against the ALIGN baseline. An MLP variant $3d \to h \to d$ is an ablation and, if run, must state h, the activation, dropout, and its own parameter count.

$$g_i \;=\; \sigma\big(W_g\,[\,e_i;\, t_i;\, l_i\,] + c_g\big) \in (0,1)^d \qquad z_i \;=\; g_i \odot f_i + (1 - g_i) \odot e_i$$

The vector z_i replaces the ordinary input embedding. All transformer layers above remain frozen.

What is replaced, and what is not. z_i substitutes for the *word* embedding only. Position embeddings and token-type embeddings are supplied by the encoder as usual — in practice, by passing z through the model's `inputs_embeds` interface. Adding position encodings inside the module would double-count them, and the failure is silent: the model trains, the loss decreases, and every number is wrong. This is the single most likely implementation bug in the project.

What the gate actually guarantees. An earlier version of this document claimed that $g_i \to 0$ recovers the unmodified model exactly. That claim was false, and the error is worth stating plainly because it changes how the whole design should be described.

Since $e_i = \mathrm{Emb}_\theta(b_i)$ is computed from the *stripped* base stream, $g_i \to 0$ yields $E_\theta\big(T(b(x))\big)$, not $E_\theta\big(T(x)\big)$. The gate recovers the **base-only pathway**, not the original model. Whether clean-input performance survives that substitution is not a structural guarantee at all — it is exactly hypothesis H1, and exactly what G1 measures.

Consequence: the base grid is invariant by construction. Restating the design honestly makes it look better, not worse. Because $b(x) = b(\tilde{x})$ for every corruption rate, UNMARK sees the *same* token grid whatever the input condition. Corruption is therefore, inside UNMARK, a purely *channel-level* phenomenon: only τ changes. For FLOOR, RESTORE and ALIGN, corruption is a string-level phenomenon that re-tokenizes the input and changes its length. That difference is the substance of the contribution, and it should be the framing in the paper: UNMARK does not repair a damaged input, it removes the input's dependence on whether the damage occurred.

The cost is equally real and must be reported: even on fully diacritized text, UNMARK *forgoes* the encoder's original tokenization and must reconstruct what it needs from channels.

Consequence for the experimental tables. UPPER (unmodified model, clean text) and UNMARK on clean text are *different input pathways*, not the same pathway with a module attached. The "no clean-input regression" control is therefore a cross-pathway comparison, and the paper must say so rather than implying an identity that does not hold.

Gating on a frozen encoder is standard practice and is *not* claimed as novel; it is included for adaptive behaviour and must be cited accordingly.

4.6 Training objectives

Training is self-supervised and requires no annotation. Any Vietnamese corpus suffices; corrupted counterparts are generated by rule.

Structured channel dropout. For each example, sample a corruption rate $p \sim \mathcal{U}(0, 1)$ and set the tone channel to UNMARKED for a random p-fraction of syllables. Sampling p continuously, rather than using a fixed rate or only the endpoints, is essential: a model trained only at $p = 0$ and $p = 1$ will behave poorly at intermediate rates, which are the realistic ones. An optional second rate governs letter-diacritic dropout.

Alignment loss. Match the corrupted representation to the clean one produced by the same frozen encoder without the module:

$$\mathcal{L}_{\text{align}} \;=\; D\big(h'(\tilde{x}_p),\, h(x)\big)$$

Locked for v1: pooled representations only, with D the cosine distance. Per-token alignment is deferred. The reason is stated in Section 4.4: the clean and corrupted strings do not share a token grid, so per-token alignment is itself an alignment problem, and it should not be solved before it is known whether the module works at all. Per-token alignment is an extension, not part of v1.

Clean-preservation loss. Force near-identity behaviour on uncorrupted input:

$$\mathcal{L}_{\text{clean}} \;=\; D\big(h'(x),\, h(x)\big)$$

Total. $\mathcal{L} = \lambda_a \mathcal{L}_{\text{align}} + \lambda_c \mathcal{L}_{\text{clean}}$, with λ_a, λ_c tuned on a development split. The measurable training objective, stated conservatively, is

$$D\big(h_{\text{UNMARK}}(x), h_{\text{UNMARK}}(\tilde{x})\big) \;<\; D\big(h_{\text{base}}(x), h_{\text{base}}(\tilde{x})\big).$$

4.7 Parameter budget

Component	Parameters
Tone embedding table ($7 \times d$)	$\approx$ 5K
Letter-diacritic table ($\sim$10 $\times d$, per character)	$\approx$ 8K
Fusion projection ($3d \to d$)	$\approx$ 1.8M
Gate projection ($3d \to d$)	$\approx$ 1.8M
Layer norms	$\approx$ 2K
Total trainable	$\approx$ **3.6M**
Frozen encoder	135M+

A linear fusion variant reduces this further and is included in the ablation.

5 Specification lock

Status: partially locked. The architecture is locked. Several concrete values are still open, and each one blocks a specific gate:

RESTORE checkpoint and decoding parameters	blocks G2
Dataset versions and splits	blocks G2 and the full grid
Stage-1 corpus	blocks stage-1 training
ALIGN architecture and budget	blocks G3
Classification head concrete values	blocks G1

Work that depends on none of these — orthography, corruption, alignment, tests — can begin immediately. Calling this section "locked" before the table above is filled in would be self-deception.

Everything below is frozen before the gates begin. The purpose is to prevent the failure mode in which architecture and experimental protocol are adjusted while results are being read — which turns a study into trial and error and makes the comparison uninterpretable.

The rule is not "never change anything". G1 exists precisely to discover whether the architecture survives contact with a frozen encoder. The rule is: **whatever enters the comparison is locked, and whatever is changed is logged with the reason and the date.**

5.1 Architecture (locked before G1)

Item	Locked value
Base stream	stripped text, frozen tokenizer, frozen embedding table
Tone channel	7 slots (5 marked + 2 policy slots), syllable level
Letter channel	closed set, **character** level, mean-pooled in embedding space
Main fusion	single linear projection, $3d \to d$ (MLP is an ablation)
Gate	per-dimension, $\sigma(W_g[\cdot])$
Normalisation	LayerNorm after fusion, before the gate combination
Position / token-type	supplied by the encoder via `inputs_embeds`
Encoder	fully frozen; no layer unfrozen without a logged decision
Alignment loss	pooled, cosine distance
Corruption sampling	$p \sim \mathcal{U}(0,1)$ per example, continuous
Stage-2 head	trained on **clean data only**, then frozen (see below)

Linear-versus-MLP fusion, gate-versus-no-gate, and tone-only-versus-both-channels are *ablations*, not open architecture questions.

5.2 Baselines (locked before any UNMARK number is seen)

This is a scientific-integrity requirement, not a convenience. If baselines are tuned after UNMARK's score is known, the comparison is worthless.

- RESTORE: named restorer, pinned checkpoint and version, frozen, fixed decoding parameters. Its own token-level accuracy is measured and reported separately, on long text and on short text.
- ALIGN: architecture, parameter budget matched to UNMARK within a stated tolerance, loss, and training budget — all fixed in advance.
- **Classification head**: one architecture, identical across all five systems, with the same learning-rate schedule, epoch budget, early-stopping criterion, and seed list. The concrete values (hidden size, pooling, learning rate, epochs, patience) are pinned during spec lock; "identical" is not a specification until the numbers are written down.

The head is trained on clean data only. This is a substantive experimental decision, not a detail. Two protocols are possible:

Protocol	What it measures
Head trained on clean only, then frozen and evaluated on every condition	Whether the *representation* is stable under corruption.
Head trained on clean plus corrupted data	Whether the *system* is robust — but robustness now comes partly from supervised noise augmentation in the head.

UNMARK's research question is about representation stability, so the first protocol is locked. A reviewer can otherwise attribute any gain to the head having seen corrupted labels, and the claim

about input representations would not follow from the evidence. The same protocol applies to all five systems.

5.3 Data and corruption (locked before G2)

Each task is pinned by dataset name, version, split, label mapping, metric, maximum sequence length, and normalisation. Corruption is a deterministic function

$$C(x, p, s) \longrightarrow \tilde{x}$$

of example, rate, and seed: the same triple must always produce the same corrupted string. If corruption is nondeterministic, RESTORE, ALIGN, and UNMARK are silently evaluated on different noise, and the whole results table becomes meaningless without any error being raised.

5.4 Split discipline

Split	Permitted use
train	module training, head training
dev	every architecture and hyperparameter decision, all ablations
test	one final evaluation, for the tables that appear in the paper

When results are being read under time pressure, the risk of drifting into decisions made on test is high. Ablations are reported on dev; only final tables use test.

6 Experimental design

6.1 Backbones

At least two, to show the module is not tied to one model: PhoBERT-base (word/syllable-level BPE, trained on formal diacritized text) and ViSoBERT (social-media oriented, already partially robust). A multilingual encoder such as XLM-R is a third option if time allows. Including ViSoBERT is deliberate: it is the hardest case, since it is already the most robust baseline available, and a gain there is the most convincing result obtainable.

6.2 Tasks

Classification, not retrieval — substantially cheaper and sufficient to test every hypothesis. Use the task suite on which Vietnamese diacritic degradation has already been reported, so numbers are comparable to published values: emotion recognition, hate-speech detection, sentiment analysis, and spam-review detection.

6.3 Corruption conditions

Condition	Description
FULL	Fully diacritized (upper bound)
P25 / P50 / P75	Tone marks removed from 25/50/75% of syllables
P100	All tone marks removed
STRIP-ALL	Tone and letter diacritics removed (real typing behaviour)
VARIANT	Tone-placement variants (hoà/hòa) and NFC/NFD forms

STRIP-ALL is the condition that matches how people actually type and should be reported as the headline number. VARIANT is cheap to add and tests a failure mode nobody has measured.

6.4 Systems compared

System	Level	Description
UPPER	—	Clean input, unmodified model
FLOOR	—	Corrupted input, unmodified model
RESTORE	String	Off-the-shelf diacritic restorer, then encode
ALIGN	Output	Contrastive alignment adapter on the encoder output
UNMARK	Input	This proposal

All five share the backbone, the classification head architecture, the hyperparameters, the seeds, and the training budget. Any deviation makes the comparison uninterpretable.

6.5 Metrics

Task level. Macro-F1 and accuracy per task and condition.

Gap Recovery Rate. The headline number, normalising across tasks of different difficulty:

$$\mathrm{GRR} = \frac{S_{\mathrm{system}} - S_{\mathrm{FLOOR}}}{S_{\mathrm{UPPER}} - S_{\mathrm{FLOOR}}}$$

Reported for RESTORE, ALIGN, and UNMARK. The comparison of these three GRR values is the result of the paper.

Representation level. Cosine similarity between clean and corrupted representations, before and after the module; the distribution shift; and a low-dimensional projection showing convergence of the two populations. This is what distinguishes an analysis from a leaderboard entry.

Cost. Added parameters, encoding latency, and whether the pipeline requires an additional model at inference. RESTORE carries an autoregressive decoder; UNMARK carries a matrix multiplication.

Diagnostic. Restorer token-level accuracy measured separately on long text and on short, context-poor input, and error rate broken down by word class (proper nouns and domain terms versus function words). If the predicted pattern holds — errors concentrated in the high-information words — this is a single figure that motivates the entire paper.

6.6 Controls

- **No clean-input regression.** Every system must be reported on FULL. A system that improves corrupted input while degrading clean input is useless in deployment, where the mode is unknown. Note that for UNMARK this is a *cross-pathway* comparison — UPPER runs the encoder's own tokenization, UNMARK runs the base grid — and the paper must state that rather than implying an identity.
- **Seeds.** At least three per configuration; report mean and standard deviation. If the improvement is within seed variance, there is no result, and this must be stated.
- **Ablations.** Linear versus MLP fusion; with versus without gate; tone channel only versus tone plus letter channel; training data volume; fixed versus sampled corruption rate. The H4 ablation is separated out below because it carries the originality of the work.
- **LLM reference point.** A zero-shot general-purpose LLM row, included as a reference rather than a competing baseline, to pre-empt the "why not just use an LLM" question and to test whether the problem disappears at scale.

6.7 The H4 experiment: three tone-state policies

H4 is the hypothesis that carries whatever originality this work has, so it is specified as a three-way comparison rather than a binary ablation. All three share everything except how an unmarked syllable is labelled.

Policy	Label assigned when no tone mark is visible	Deployable
FORCED-NGANG	always *ngang* — treats absence as confident information	yes
OBSERVABLE	always UNMARKED — ambiguity is represented, **this is UNMARK**	yes
ORACLE	*ngang* if genuinely *ngang*, MISSING if stripped	**no**

ORACLE uses knowledge available only because corruption is synthetic. It **cannot be used at inference** and exists purely as an upper-bound diagnostic. The paper must say so explicitly wherever the number appears.

The predicted ordering is

$$\text{FORCED-NGANG} < \text{OBSERVABLE} \leq \text{ORACLE}.$$

If it holds, the paper gets two results at once: modelling the ambiguity helps, and the remaining distance to ORACLE quantifies how much ambiguity is still unresolved. If OBSERVABLE approaches ORACLE, that is the strongest available outcome — it means context resolves nearly all of it. If FORCED-NGANG matches OBSERVABLE, H4 is false and the paper must say so.

6.8 Cheap diagnostics, run before the full grid

Three measurements that cost little and pay for themselves in explanatory power.

Tokenization fragmentation. Subwords per syllable, per corruption condition, per backbone. This quantifies exactly what UNMARK is up against, and whether backbones differ in how badly they fragment undiacritized input.

Representation drift. $\cos\big(h(x), h(\tilde{x})\big)$ before the module versus $\cos\big(h(x), h_{\text{UNMARK}}(\tilde{x})\big)$ after. This is mechanistic evidence, and it holds even if downstream gains are modest.

Ambiguity subset. Partition the test data by whether it contains syllables from large collision groups. If UNMARK improves far more on the ambiguous partition than on the ordinary one, that is direct evidence for H4 — evidence of mechanism, not merely of outcome.

7 Feasibility gates

These run *before* any writing. Their purpose is to fail fast.

G0 — Decomposition correctness (Day 1, no GPU). Implement `decompose` / `recompose` with the character-level letter channel, and run round-trip on ≥100K sentences.
Pass: $\text{rec}(\text{dec}(x)) = \text{canon}(x)$ on 100% of inputs, where the only permitted differences are NFC normalisation and tone-mark placement — *and every such difference is enumerated in a report, not silently absorbed.* Letter diacritics must reconstruct exactly, with no exceptions.
Test corpus must include: ordinary Vietnamese; NFC and NFD forms; mixed Vietnamese/English; digits; punctuation; URLs and e-mail addresses; emoji; đ/Đ; all of ă â ê ô ơ ư; all six tones; syllables carrying multiple modified vowels; tone-placement variants; uppercase and title case; and empty or whitespace-only strings.
Fail: fix before anything else. Nothing is trained until G0 passes.
G1 — Frozen encoder acceptance (Day 2). Attach the fusion layer, train briefly on a small corpus, force the gate towards identity, evaluate on one classification task with FULL input.
Pass: within ≈1 point of the unmodified model.

Fail: the encoder rejects the synthesised embedding distribution. Options: unfreeze the lowest layers (loses the retrofit claim), or abandon the input-level design. Either way, know it on day 2.
G2 — Residual gap after restoration (Day 3). Run UPPER, FLOOR, and RESTORE on two tasks.
Pass: a clear gap remains between RESTORE and UPPER.
Fail: if restoration nearly closes the gap, H2 is false. The paper becomes "a cheaper route to equivalent performance" — publishable but much weaker — or the project falls back to the analysis-only version (Section 12).
G3 — Pilot (Day 4–5). Not a gate on feasibility but a gate on *signal*. One backbone, one task, three conditions (FULL, P50, STRIP-ALL), four systems (FLOOR, RESTORE, ALIGN, UNMARK), one seed.
Pass: UNMARK shows a visible advantage over ALIGN on the corrupted conditions while holding FULL steady.
Caveat: one seed is a go/no-go signal, *not* evidence. It can mislead in either direction, and the temptation to believe it is strong. Nothing from the pilot appears in the paper without the full seed set behind it.

The full grid — 5 systems × tasks × conditions × 3 seeds × 2 backbones — is launched only after G3. Running it earlier risks spending the entire compute budget discovering something G1 would have revealed in an afternoon.

The sequence is deliberate:

$$\text{SPEC LOCK} \rightarrow \text{G0} \rightarrow \text{G1} \rightarrow \text{G2} \rightarrow \text{PILOT} \rightarrow \text{FULL GRID}$$

Five days to decide whether to invest the remaining sixteen. This is the best trade available in the project.

8 Implementation plan

8.1 Repository layout

```
unmark/
  README.md
  requirements.txt
  configs/
    base.yaml # backbone, paths, seeds
    train_module.yaml # stage 1: self-supervised module training
    train_head.yaml # stage 2: task head training
    ablations/ # one file per ablation
  unmark/
    __init__.py
    orthography/
      decompose.py # dec / rec, channel state sets
      corrupt.py # structured channel dropout
      variants.py # NFC/NFD, tone-placement variants
      tables.py # syllable inventory, mark inventories
    align/
      offsets.py # syllable -> subword label propagation
    modules/
      channels.py # tone & letter embedding tables
      fusion.py # fusion projection + gate
      unmark.py # wraps a frozen encoder
    baselines/
      restore.py # string-level restoration wrapper
      align_adapter.py # output-level alignment adapter
    training/
      stage1_module.py
      stage2_head.py
      losses.py
    eval/
      metrics.py # macro-F1, GRR
      representation.py # cosine, shift, projection
```

```
      cost.py # latency, parameter counts
      report.py # builds result tables
  scripts/
    g0_roundtrip.py # gate 0
    g1_acceptance.py # gate 1
    g2_restore_gap.py # gate 2
    g3_pilot.py # pilot: 1 backbone, 1 task, 1 seed
    diagnostics.py # fragmentation, drift, ambiguity subset
    run_grid.py
  tests/
    test_decompose.py # round-trip, edge cases
    test_offsets.py # alignment invariants
  results/
  paper/
```

8.2 Core interfaces

```
# unmark/orthography/decompose.py
@dataclass
class Decomposition:
    base: str # all diacritics removed
    tone: list[int] # PER SYLLABLE; 0 = UNMARKED
    letter: list[int] # PER CHARACTER of `base`; 0 = NONE
    spans: list[tuple[int,int]] # char span of each syllable in `base`
    # invariant: len(letter) == len(base); len(tone) == len(spans)

def decompose(text: str) -> Decomposition: ...
def recompose(d: Decomposition) -> str: ... # == canon(text); see G0
def canon(text: str) -> str: ... # NFC + fixed tone placement

# unmark/orthography/corrupt.py
def corrupt(text: str, p: float, seed: int) -> str:
    \"\"\"Deterministic: the same (text, p, seed) ALWAYS yields the same output.\"\"\"

def strip_tones(d: Decomposition, p: float, rng) -> Decomposition: ...
def strip_letters(d: Decomposition, p: float, rng) -> Decomposition: ...

# unmark/align/offsets.py
def propagate(d: Decomposition, tokenizer) -> tuple[list[int], list[int], list[int]]:
    """Return (input_ids, tone_ids, letter_ids) on the BASE token grid."""

# unmark/modules/unmark.py
class UnmarkEncoder(nn.Module):
    """Frozen encoder + trainable input-side module."""
    def __init__(self, encoder, d_model, n_tone=7, n_letter=10,
                 fusion="linear", use_gate=True,
                 tone_policy="observable"): # observable | forced_ngang | oracle
        # n_tone = 5 marked + 2 policy slots, identical for all three policies
        ...
    def forward(self, input_ids, tone_ids, letter_ids, attention_mask):
        z = self.fuse(input_ids, tone_ids, letter_ids)
        # word embeddings ONLY: position/token-type come from the encoder
        return self.encoder(inputs_embeds=z, attention_mask=attention_mask)
```

8.3 Two-stage training

1. **Stage 1 — module.** Self-supervised. Encoder frozen. Train ϕ with $\mathcal{L}_{\text{align}} + \mathcal{L}_{\text{clean}}$ under sampled corruption. No labels required. Corpus: any Vietnamese text.
2. **Stage 2 — head.** Module frozen, encoder frozen. Train a classification head per task **on clean data only**, then freeze it and evaluate under every corruption condition. Run identically for all five systems.

Keeping the stages separate is what makes the comparison fair: only the input representation differs between systems at stage 2.

8.4 Compute

One GPU. Stage 1 is a few hours; the full stage-2 grid (5 systems × tasks × conditions × 3 seeds) is the larger cost and should be scripted, checkpointed, and resumable from the start. Colab or Kaggle is sufficient.

9 Timeline

There is no submission deadline attached to this project. The schedule below is a *working* plan sized to roughly four weeks from 19 August 2026, kept because a project without a rhythm drifts — not because anything must be handed in.

Two consequences of decoupling from a deadline are worth stating. The good one: if a gate fails, there is room to fix the design properly instead of shipping whatever runs. The bad one: the discipline that a deadline supplies must now come from the gates themselves, so the pass/fail criteria in Section 7 should be treated as binding rather than advisory.

Dates	Phase	Deliverable
Aug 19	Spec lock	Section 5 filled in; baselines pinned
Aug 20–22	Gates	G0, G1, G2 passed or project redirected
Aug 23–24	Pilot	G3: one backbone, one task, one seed — go/no-go
Aug 25–29	Build	Both baselines finalised, stage-1 training, diagnostics
Aug 30–Sep 3	Grid	Full grid, 3 seeds, 2 backbones
Sep 4–6	Analysis	Ablations incl. the three-policy H4 experiment
Sep 7–10	Draft	Full draft of the paper; all figures and tables final
Sep 11–14	Revise	Related work hardened; every claim audited against Section 3
Sep 15–16	Slack	Reserved for what goes wrong
Sep 16	**Milestone**	**Complete, venue-independent draft**
Then	Decide	Choose a venue that fits the strength of the result, and reformat to its template

The slack at the end is not optional and should not be spent in advance. Its purpose is to absorb the failure that has not happened yet.

10 Risks and mitigations

Risk	Sev.	Mitigation
Frozen encoder rejects synthesised embeddings	High	G1 on day 2; fallback to unfreezing lowest layers or to the analysis paper
Restoration closes the gap (H2 false)	High	G2 on day 3; pivot the story to cost, or fall back
Reviewer reads the work as "2SDiac for Vietnamese"	High	Confront it in the introduction; H4 ablation is the concrete rebuttal
Machinery overlaps PhonoSTFG (frozen PLM + gate)	Medium	Cite in the method section, not only in related work
Gains within seed variance	Medium	Three seeds from the start; report honestly if so
Alignment bug between streams	Medium	Unit tests and invariants before training (Section 4.4)
Position embeddings added twice	Medium	Word embeddings only, via `inputs_embeds`; assert in a unit test
Base grid discards useful original tokenization on clean input	High	Acknowledged cost, not a bug; H1 and G1 quantify it before anything is built on top
Head trained on corrupted data confounds the claim	Medium	Clean-only head protocol locked in Section 5
Nondeterministic corruption across systems	Medium	$C(x, p, s)$ pinned by seed; regression test on fixed examples
Decisions drift onto the test split	Medium	Ablations on dev; test opened once (Section 5)
Over-reading the one-seed pilot	Medium	Pilot is go/no-go only; nothing from it enters the paper
Unicode edge cases (NFC/NFD, mixed script)	Medium	G0 round-trip on 100K sentences
Concurrent Vietnamese work not indexed in English	Medium	Search Vietnamese-language sources and national conference proceedings early
Scope creep into decoding-side work	Low	Explicitly out of scope; a neighbouring group has flagged that direction

11 Reproducibility and artifact

The module is the artifact, and it only becomes one if released properly. Plan from day one, not at submission:

- Public repository with code, configs, and seeds.
- Trained module weights for each backbone — a few megabytes.
- The decomposition and corruption utilities as a standalone, importable component, so others can reuse them without adopting the model.
- A minimal usage example: load a frozen encoder, wrap it, run inference.
- Exact dataset versions and splits; all preprocessing scripted.
- A results directory containing raw per-seed numbers, not only aggregates.

12 Fallback: the analysis-only paper

If G1 or G2 fails, the project pivots rather than dies. The fallback paper:

> A controlled study of how existing robustness strategies behave under Vietnamese diacritic loss — a structured, ambiguous form of orthographic degradation that differs from the character noise these methods were designed for. String-level restoration, output-level alignment, and augmentation-based robustness are compared across corruption rates, backbones, and tasks, with error analysis by word class and a diagnosis of where each strategy breaks.

This is honest, feasible in the remaining time, and reuses everything already built. It is a weaker result than the method paper, and it should be described as such rather than dressed up — but it is a real contribution and a genuine option, not a failure state.

13 Open decisions

To be settled during the gate phase and recorded here as they are made:

Most of what was open in v1.0 is now locked in Section 5. What remains, to be settled during spec lock and recorded here with a date:

1. Which diacritic restorer serves as RESTORE: name, checkpoint, version, decoding parameters.
2. Exact dataset versions and splits for the four tasks.
3. Corpus for stage-1 training: size, domain mix, and whether it should match the downstream task domains.
4. ALIGN baseline: concrete architecture, parameter budget, loss, and training budget.
5. Classification head: hidden size, pooling, learning rate, epoch budget, early-stopping patience, seed list.
6. Whether to include a retrieval task if time permits, or stay entirely within classification. (Default: stay.)

Changelog

v1.3 (19 Aug 2026). Corrected a false structural claim: the gate recovers the *base-only pathway*, not the unmodified model, because e_i is computed from the stripped stream. The consequences are now stated positively (the base grid is invariant by construction, so corruption is channel-level inside UNMARK) and negatively (clean input forgoes the encoder's original tokenization; UPPER is a different pathway). Tone table enlarged to 7 slots so all three H4 policies share one architecture. Main fusion fixed as a single linear projection, resolving the disagreement between the architecture lock and the parameter budget. Letter channel pools in embedding space rather than label space. Stage-2 head locked to clean-only training. Deterministic rule added for deciding whether an alphabetic span is Vietnamese. Section 5 relabelled *partially* locked, with a table of what each open item blocks.

v1.2 (19 Aug 2026). All venue and submission-deadline references removed. The schedule is a working plan rather than a countdown, and the gates carry the discipline the deadline used to supply.

v1.1 (19 Aug 2026). Letter-diacritic channel moved from syllable level to **character** level, which is what makes reconstruction achievable; G0 restated as canonical reconstruction with all differences enumerated, rather than byte-for-byte equality; H4 expanded into a three-policy experiment with a non-deployable oracle upper bound; alignment loss locked to pooled-only for v1; `inputs_embeds` requirement made explicit; Section 5 (specification lock) added; pilot stage G3 inserted before the full grid; three cheap diagnostics added; split discipline stated.

References

[1] P. Bahar, M. Di Gangi, N. Rossenbach, M. Zeineldeen. Take the Hint: Improving Arabic Diacritization with Partially-Diacritized Text. *Interspeech*, 2023.

[2] N. Q. Nguyen et al. ViSoBERT: A Pre-Trained Language Model for Vietnamese Social Media Text Processing. *EMNLP*, 2023.

[3] D. Q. Nguyen, A. T. Nguyen. PhoBERT: Pre-trained Language Models for Vietnamese. *Findings of EMNLP*, 2020.

[4] N. N.-Y. Nguyen, A.-D. Nguyen, N. H. Nguyen, K. V. Nguyen, N. L.-T. Nguyen. Linguistically Informed Multimodal Fusion for Vietnamese Scene-Text Image Captioning: Dataset, Graph Framework, and Phonological Attention. arXiv:2604.27712, 2026.

[5] Noise-Robust Dense Retrieval via Contrastive Alignment Post Training, 2023.

[6] Typo-Robust Representation Learning for Dense Retrieval. *ACL*, 2023.

[7] S. Zhuang, G. Zuccon. Characterbert and Self-Teaching for Improving the Robustness of Dense Retrievers on Queries with Typos. *SIGIR*, 2022.

[8] Z. Sun et al. ChineseBERT: Chinese Pretraining Enhanced by Glyph and Pinyin Information. *ACL*, 2021.

[9] Interplay of Machine Translation, Diacritics, and Diacritization. *NAACL*, 2024.

[10] L. S. T. Nguyen, T. T. Quan. Which Works Best for Vietnamese? A Practical Study of Information Retrieval Methods across Domains. *Findings of EACL*, 2026.

Citations above are working entries recorded during the literature survey. Full bibliographic details must be verified against the official proceedings before submission.